HW 01 — Floating-Point Arithmetic and Numerical Derivatives

Part A: Powers of the Golden Ratio Computed Downward

Problem Statement

Define the function `golden_powers_down(n, type)` that returns an `n×3` NumPy array (in `float32` or `float64`) comparing two ways to compute the inverse powers ϕ^{-k} for $k = 0, 1, \ldots, n-1$:

1. **Linear recurrence** column: $g_k = g_{k-2} - g_{k-1}$, seeded with $g_0 = 1$, $g_1 = 1/\phi$.
2. **Direct** column: $g_k = \phi^{-k}$ computed by Python's `**` operator.
3. **Relative difference** column: $(g_k^{\text{linear}} - g_k^{\text{direct}})/g_k^{\text{direct}}$.

Run for $n = 31$, `float32`. Plot the $\log_{10}$ of the absolute relative error vs. k.

Mathematical Background

The golden ratio $\phi = (1 + \sqrt{5})/2 \approx 1.618$ satisfies $\phi^2 = \phi + 1$. Dividing both sides by ϕ^n gives a recurrence for $g_n = \phi^{-n}$:

$$g_n = g_{n-2} - g_{n-1}$$

Verification: $g_{n-2} - g_{n-1} = \phi^{-(n-2)} - \phi^{-(n-1)} = \phi^{-n}(\phi^2 - \phi) = \phi^{-n} \cdot \phi(\phi - 1) = \phi^{-n}$, since $\phi - 1 = 1/\phi$. □

The recurrence is seeded at $g_0 = 1$, $g_1 = 1/\phi \approx 0.618$.

Why the Recurrence Is Numerically Unstable

The general solution to $g_n = g_{n-2} - g_{n-1}$ is a linear combination of **two** basis solutions. The characteristic equation $r^2 + r - 1 = 0$ has roots $r_1 = \phi^{-1}$ and $r_2 = -\phi$, so:

$$g_n = A \cdot \phi^{-n} + B \cdot (-\phi)^n$$

In exact arithmetic with $g_0 = 1$, $g_1 = 1/\phi$, the coefficient $B = 0$, giving the correct decaying sequence $\phi^{-n} \to 0$.

In **floating-point**, the seeds g_0 and g_1 are slightly rounded, introducing a tiny $B \neq 0$. The spurious component $B(-\phi)^n$ grows like $\phi^n \to \infty$ while the true answer $\phi^{-n} \to 0$. The relative error therefore grows as:

$$\frac{|B(-\phi)^n|}{\phi^{-n}} = |B| \cdot \phi^{2n}$$

Each additional step multiplies the relative error by $\phi^2 \approx 2.618$. In log scale the error grows linearly with slope $2\log_{10}(\phi) \approx 0.42$ per step.

Algorithm and Implementation

```
def golden_powers_down(n, type):
    dt = np.dtype(type)                # float32 or float64
    result = np.zeros((n, 3), dtype=dt)
    phi = dt.type((1.0 + np.sqrt(5.0)) / 2.0)  # cast phi to the target precision

    result[0, 0] = result[0, 1] = dt.type(1.0)          # g_0 = 1
    result[1, 0] = result[1, 1] = dt.type(1.0 / phi)    # g_1 = 1/phi

    for i in range(2, n):
        linear = dt.type(result[i-2, 0] - result[i-1, 0])  # g_{n-2} - g_{n-1}
        direct = dt.type(phi ** -i)                         # phi^{-i}
        rel_diff = dt.type((linear - direct) / direct)
        result[i] = [linear, direct, rel_diff]
    return result
```

Key detail: the dtype cast `dt.type(...)` is applied at every arithmetic operation so that all intermediate values are stored and rounded in `float32` (not promoted to `float64` silently). This faithfully reproduces the rounding errors that accumulate in restricted-precision hardware.

How It Works and What the Plot Shows

- For small k (roughly 0–7), the recurrence matches the direct formula to `float32` machine precision ($\sim 10^{-7}$), so errors scatter near -7 in log scale.
- From $k \approx 10$, the log error grows linearly with the predicted slope ≈ 0.42.
- At $k \approx 20$, the relative error exceeds 100%—the recurrence result bears no resemblance to the true value.
- The `RuntimeWarning: divide by zero` in the log plot occurs because the relative error is exactly 0 for some early k (the two methods agree to full `float32` precision), making $\log_{10}(0) = -\infty$.

Key principle illustrated: a mathematically valid recurrence can be computationally useless if it amplifies rounding errors. The physical cause here is *catastrophic cancellation*: subtracting two nearly equal small numbers amplifies the rounding error to dominate the result.

Part B: Numerical Derivatives of sin(x)

Problem Statement

Compare three finite-difference formulas for approximating $f'(x)$ where $f(x) = \sin(x)$:

Method	Formula	Truncation error order
2-point (forward)	$\frac{f(x+h)-f(x)}{h}$	$O(h)$
3-point (central)	$\frac{f(x+h)-f(x-h)}{2h}$	$O(h^2)$
5-point (central)	$\frac{-f(x+2h)+8f(x+h)-8f(x-h)+f(x-2h)}{12h}$	$O(h^4)$

Table 1: Compare errors at 10 random x values with $dx = 0.002$.
Table 2: Scan dx from 10^{-1} to 10^{-10} at $x = 0.7$.

Algorithm: Taylor-Series Derivation

All three formulas are derived by expanding $f(x \pm h)$ in a Taylor series around x:

$$f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2}f''(x) \pm \frac{h^3}{6}f'''(x) + \frac{h^4}{24}f^{(4)}(x) \pm \cdots$$

- **2-point:** $[f(x+h) - f(x)]/h = f'(x) + \frac{h}{2}f''(x) + O(h^2)$. The error (approximation minus true value) is $\approx +\frac{h}{2}f''(x)$, i.e., $O(h)$.
- **3-point:** Subtracting the expansion of $f(x-h)$ from that of $f(x+h)$ cancels the even-power terms: $[f(x+h)-f(x-h)]/(2h) = f'(x)+\frac{h^2}{6}f'''(x)+O(h^4)$. Error $\approx +\frac{h^2}{6}f'''(x)$, i.e., $O(h^2)$.
- **5-point:** Uses $f(x\pm h)$ and $f(x\pm 2h)$ with carefully chosen weights to cancel both the h^2 and h^3 error contributions: $[...]/(12h) = f'(x)-\frac{h^4}{30}f^{(5)}(x)+O(h^6)$. Error $\approx -\frac{h^4}{30}f^{(5)}(x)$, i.e., $O(h^4)$.

Implementation

```
def derivative2p(f, x, dx):
    return (f(x + dx) - f(x)) / dx

def derivative3p(f, x, dx):
    return (f(x + dx) - f(x - dx)) / (2 * dx)

def derivative5p(f, x, dx):
    return (-f(x + 2*dx) + 8*f(x + dx) - 8*f(x - dx) + f(x - 2*dx)) / (12 * dx)
```

How It Works: Explaining the Two Tables

Table 1 — Why 3-point and 5-point errors are constant across all x values:

For $f(x) = \sin(x)$, derivatives cycle: $f' = \cos x$, $f'' = -\sin x$, $f''' = -\cos x$, $f^{(5)} = \cos x$.

The *relative* error (error divided by $f'(x) = \cos x$) for the 3-point formula is:

$$\text{rel. err}_{3p} = +\frac{h^2}{6} \cdot \frac{f'''(x)}{f'(x)} = +\frac{h^2}{6} \cdot \frac{-\cos x}{\cos x} = -\frac{h^2}{6}$$

This is **independent of x**, which is why all 10 rows show the same `err3p` $= -h^2/6 \approx -6.67 \times 10^{-7}$ at $h = 0.002$ (note the printed values are negative). Similarly:

$$\text{rel. err}_{5p} = -\frac{h^4}{30} \cdot \frac{f^{(5)}(x)}{f'(x)} = -\frac{h^4}{30} \cdot \frac{\cos x}{\cos x} = -\frac{h^4}{30} \approx -5.3 \times 10^{-13}$$

The 2-point relative error $\approx \frac{h}{2}\frac{f''(x)}{f'(x)} = -\frac{h}{2}\tan(x)$ **does** vary with x (and is negative for $x \in (0, 1)$, matching the table).

Table 2 — Two competing error sources:

As dx decreases, two effects compete:

1. **Truncation error** decreases (theory says $O(h^p)$).
2. **Round-off error** increases because subtracting nearly-equal floats loses significant digits.

The total error is minimized at an optimal dx where these balance. Estimating with machine epsilon $\varepsilon \approx 2.2 \times 10^{-16}$:

Method	Optimal dx estimate	Observed minimum
2-point	$\sqrt{2\varepsilon} \approx 2.6 \times 10^{-8}$	between 10^{-7} and 10^{-8}
3-point	$(3\varepsilon)^{1/3} \approx 9.5 \times 10^{-6}$	between 10^{-5} and 10^{-6}
5-point	$(45\varepsilon/4)^{1/5} \approx 1.3 \times 10^{-3}$	between 10^{-3} and 10^{-4}

Key insight: Higher-order methods are more accurate at moderate dx but hit round-off *sooner* (at larger dx), because their stencils involve more cancellation. For $dx < 10^{-8}$, all methods are dominated by floating-point noise.

Key Takeaways

- **A mathematically exact recurrence can be numerically useless.** $g_n = g_{n-2} - g_{n-1}$ is exact on paper, but in finite precision it amplifies a spurious growing solution $B(-\phi)^n$, so the relative error grows like ϕ^{2n}. Always ask whether a recurrence amplifies or damps rounding errors (check the roots of its characteristic equation).
- **Catastrophic cancellation** — subtracting two nearly-equal numbers — destroys significant digits and is the root cause of most floating-point instability.
- **Finite-difference accuracy is set by Taylor cancellation.** Symmetric (central) stencils cancel even-order error terms, so 2-point is $O(h)$, 3-point $O(h^2)$, 5-point $O(h^4)$.
- **There is an optimal step size.** Total error = truncation (shrinks with h) + round-off (grows as ε/h). The minimum sits where they balance; shrinking h past it makes things *worse*.
- **Higher order is not free:** higher-order stencils reach round-off-limited accuracy at a *larger* h, because they involve more cancellation.